

Test problem 2: Flow past a cylinder (DFG 2D-3 benchmark)

Print to PDF

Contents

- Mesh generation
- Generating the mesh
- Loading mesh and boundary markers
- Physical and discretization parameters
- Boundary conditions
- Variational form
- Verification of the implementation compute known physical quantities
- Solving the time-dependent problem
- Verification using data from FEATFLOW

Author: Jørgen S. Dokken

In this section, we will turn our attention to a slightly more challenging problem: flow past a cylinder. The geometry and parameters are taken from the [DFG 2D-3 benchmark](#) in

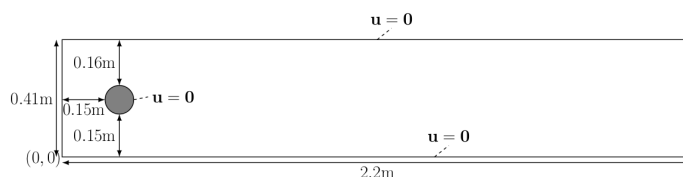
FeatFlow.

To be able to solve this problem efficiently and ensure numerical stability, we will substitute our first order backward difference scheme with a Crank-Nicholson discretization in time, and a semi-implicit Adams-Bashforth approximation of the non-linear term.

Computationally demanding demo

This demo is computationally demanding, with a run-time up to 15 minutes, as it is using parameters from the DFG 2D-3 benchmark, which consists of 12800 time steps. It is advised to download this demo and not run it in a browser. This runtime of the demo can be decreased by using 2 or 3 mpi processes.

The computational geometry we would like to use is



The kinematic viscosity is given by

$$\nu = 0.001 = \frac{\mu}{\rho} \text{ and the inflow velocity}$$

profile is specified as

$$u(x, y, t) = \left(\frac{4Uy(0.41 - y)}{0.41^2}, 0 \right)$$

$$U = U(t) = 1.5 \sin(\pi t/8)$$

which has a maximum magnitude of 1.5 at $y = 0.41/2$. We do not use any scaling for this problem since all exact parameters are known.

Mesh generation

As in the [Deflection of a membrane](#) we use GMSH to generate the mesh. We first create the rectangle and obstacle.

```
import gmsh
import os
import numpy as np
import matplotlib.pyplot as plt
import tqdm.autonotebook

from mpi4py import MPI
from petsc4py import PETSc

from basix.ufl import element

from dolfinx.fem import (
    Constant,
    Function,
    functionspace,
    assemble_scalar,
    dirichletbc,
    extract_function_spaces,
    form,
    locate_dofs_topological,
    set_bc,
)
from dolfinx.fem.petsc import (
    apply_lifting,
    assemble_matrix,
    assemble_vector,
    create_vector,
    create_matrix,
```

```

        set_bc,
    )
    from dolfinx.geometry import bb_tree,
    from dolfinx.io import VTWriter, gms
    from ufl import (
        FacetNormal,
        Measure,
        TestFunction,
        TrialFunction,
        as_vector,
        div,
        dot,
        dx,
        inner,
        lhs,
        grad,
        nabla_grad,
        rhs,
    )

    gmsh.initialize()

    L = 2.2
    H = 0.41
    c_x = c_y = 0.2
    r = 0.05
    gdim = 2
    mesh_comm = MPI.COMM_WORLD
    model_rank = 0
    if mesh_comm.rank == model_rank:
        rectangle = gmsh.model.occ.addRec
        obstacle = gmsh.model.occ.addDisk

```

```

/tmp/ipykernel_1434/3014502904.py:5:
import tqdm.autonotebook

```

The next step is to subtract the obstacle from the channel, such that we do not mesh the interior of the circle.

```

if mesh_comm.rank == model_rank:
    fluid = gmsh.model.occ.cut([(gdim
    gmsh.model.occ.synchronize()

```

To get GMSH to mesh the fluid, we add a physical volume marker

```

fluid_marker = 1
if mesh_comm.rank == model_rank:
    volumes = gmsh.model.getEntities()
    assert len(volumes) == 1
    gmsh.model.addPhysicalGroup(volume
    gmsh.model.setPhysicalName(volume

```

To tag the different surfaces of the mesh, we tag the inflow (left hand side) with marker 2, the outflow (right hand side) with marker 3 and the fluid walls with 4 and obstacle with 5. We will do this by computing the center of mass for each geometrical entity.

```

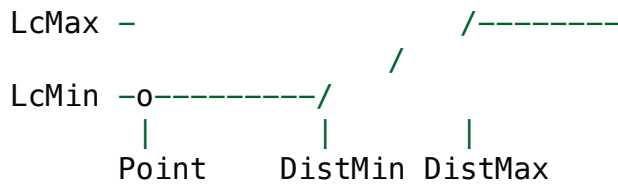
inlet_marker, outlet_marker, wall_mar
inflow, outflow, walls, obstacle = []
if mesh_comm.rank == model_rank:
    boundaries = gmsh.model.getBounda
    for boundary in boundaries:
        center_of_mass = gmsh.model.o
        if np.allclose(center_of_mass
            inflow.append(boundary[1]
        elif np.allclose(center_of_ma
            outflow.append(boundary[1
        elif np.allclose(center_of_ma
            center_of_mass, [L / 2, 0
        ):
            walls.append(boundary[1])
        else:
            obstacle.append(boundary[
    gmsh.model.addPhysicalGroup(1, wa
    gmsh.model.setPhysicalName(1, wal
    gmsh.model.addPhysicalGroup(1, in
    gmsh.model.setPhysicalName(1, infl
    gmsh.model.addPhysicalGroup(1, ou
    gmsh.model.setPhysicalName(1, out
    gmsh.model.addPhysicalGroup(1, ob
    gmsh.model.setPhysicalName(1, obs

```

In our previous meshes, we have used uniform mesh sizes. In this example, we will have variable mesh sizes to resolve the flow solution in the area of interest; close to the circular obstacle. To do this, we use GMSH

Fields.

Create distance field from obstacle. Add threshold of mesh sizes based on the distance field



```
res_min = r / 3
if mesh_comm.rank == model_rank:
    distance_field = gmsh.model.mesh.
    gmsh.model.mesh.field.setNumbers(
    threshold_field = gmsh.model.mesh
    gmsh.model.mesh.field.setNumber(t
    gmsh.model.mesh.field.setNumber(t
    gmsh.model.mesh.field.setNumber(t
    gmsh.model.mesh.field.setNumber(t
    gmsh.model.mesh.field.setNumber(t
    min_field = gmsh.model.mesh.field
    gmsh.model.mesh.field.setNumbers(
    gmsh.model.mesh.field.setAsBackgr
```

Generating the mesh

We are now ready to generate the mesh. However, we have to decide if our mesh should consist of triangles or quadrilaterals. In this demo, to match the DFG 2D-3 benchmark, we use second order quadrilateral elements.

```

if mesh_comm.rank == model_rank:
    gmsh.option.setNumber("Mesh.Algorithm")
    gmsh.option.setNumber("Mesh.Recombination")
    gmsh.option.setNumber("Mesh.Recombination")
    gmsh.option.setNumber("Mesh.Subdivision")
    gmsh.model.mesh.generate(gdim)
    gmsh.model.mesh.setOrder(2)
    gmsh.model.mesh.optimize("Netgen")

```

```

Info      : Meshing 1D...
Info      : [  0%] Meshing curve 5 (Elliptic)
Info      : [ 30%] Meshing curve 6 (Linear)
Info      : [ 50%] Meshing curve 7 (Linear)
Info      : [ 70%] Meshing curve 8 (Linear)
Info      : [ 90%] Meshing curve 9 (Linear)
Info      : Done meshing 1D (Wall 0.006s)
Info      : Meshing 2D...
Info      : Meshing surface 1 (Plane, Front)
Info      : Simple recombination completed
Info      : Simple recombination completed
Info      : Done meshing 2D (Wall 0.008s)
Info      : Refining mesh...
Info      : Meshing order 2 (curvilinear)
Info      : [  0%] Meshing curve 5 order 2
Info      : [ 20%] Meshing curve 6 order 2
Info      : [ 40%] Meshing curve 7 order 2
Info      : [ 60%] Meshing curve 8 order 2
Info      : [ 70%] Meshing curve 9 order 2
Info      : [ 90%] Meshing surface 1 order 2
Info      : Done meshing order 2 (Wall 0.008s)
Info      : Done refining mesh (Wall 0.008s)
Info      : 1952 nodes 2069 elements
Info      : Meshing order 2 (curvilinear)
Info      : [  0%] Meshing curve 5 order 2
Info      : [ 20%] Meshing curve 6 order 2
Info      : [ 40%] Meshing curve 7 order 2
Info      : [ 60%] Meshing curve 8 order 2
Info      : [ 70%] Meshing curve 9 order 2
Info      : [ 90%] Meshing surface 1 order 2
Info      : Done meshing order 2 (Wall 0.008s)
Info      : Optimizing mesh (Netgen)...
Info      : Done optimizing mesh (Wall 0.008s)

```

Loading mesh and boundary markers

As we have generated the mesh, we now need to load the mesh and corresponding facet markers into DOLFINx. To load the mesh, we follow the same structure as in [Deflection of a membrane](#), with the difference being that we will load in facet markers as well. To learn more about the specifics of the function below, see [A GMSH tutorial for DOLFINx](#).

```
mesh_data = gmshio.model_to_mesh(gmsh
mesh = mesh_data.mesh
assert mesh_data.facet_tags is not No
ft = mesh_data.facet_tags
ft.name = "Facet markers"
```

Physical and discretization parameters

Following the DGF-2 benchmark, we define our problem specific parameters

```
t = 0.0
T = 8.0 # Final time
dt = 1 / 1600 # Time step size
num_steps = int(T / dt)
k = Constant(mesh, PETSc.ScalarType(d
mu = Constant(mesh, PETSc.ScalarType(
rho = Constant(mesh, PETSc.ScalarType
```




Reduce runtime of problem

This problem takes about 15 minutes to run in serial, due to the large amount of time steps. If you convert the notebook to a python file and use `mpirun`, you can reduce the runtime of the problem.

Boundary conditions

As we have created the mesh and relevant mesh tags, we can now specify the function spaces `V` and `Q` along with the boundary conditions. As the `ft` contains markers for facets, we use this class to find the facets for the inlet and walls.

```
v_cg2 = element("Lagrange", mesh.basi
s_cg1 = element("Lagrange", mesh.basi
V = functionspace(mesh, v_cg2)
Q = functionspace(mesh, s_cg1)

fdim = mesh.topology.dim - 1
```

Define boundary conditions

```

class InletVelocity:
    def __init__(self, t):
        self.t = t

    def __call__(self, x):
        values = np.zeros((gdim, x.shape[0]))
        values[0] = (
            4 * 1.5 * np.sin(self.t *
        )
        return values

# Inlet
u_inlet = Function(V)
inlet_velocity = InletVelocity(t)
u_inlet.interpolate(inlet_velocity)
bcu_inflow = dirichletbc(
    u_inlet, locate_dofs_topological(
)
# Walls
u_nonslip = np.array((0,)) * mesh.geometry
bcu_walls = dirichletbc(
    u_nonslip, locate_dofs_topologica
)
# Obstacle
bcu_obstacle = dirichletbc(
    u_nonslip, locate_dofs_topologica
)
bcu = [bcu_inflow, bcu_obstacle, bcu_
# Outlet
bcp_outlet = dirichletbc(
    PETSc.ScalarType(0), locate_dofs_
)
bcp = [bcp_outlet]

```

Variational form

As opposed to [Pouseille flow](#), we will use a Crank-Nicolson discretization, and an semi-implicit Adams-Bashforth approximation. The first step can be written as

$$\rho \left(\frac{u^* - u^n}{\delta t} + \left(\frac{3}{2} u^n - \frac{1}{2} u^{n-1} \right) \cdot \frac{1}{2} \nabla(u \right.$$

$$u^* = g(\cdot, t^{n+1}) \quad \text{on } \partial\Omega_D$$

$$\frac{1}{2} \nu \nabla(u^* + u^n) \cdot n = p^{n-\frac{1}{2}} \quad \text{on } \partial\Omega_N$$

where we have used the two previous time steps in the temporal derivative for the velocity, and compute the pressure staggered in time, at the time between the previous and current solution. The second step becomes

$$\nabla^2 \phi = \frac{\rho}{\delta t} \nabla \cdot u^* \quad \text{in } \Omega,$$

$$\nabla \phi \cdot n = 0 \quad \text{on } \partial\Omega_D,$$

$$\phi = 0 \quad \text{on } \partial\Omega_N$$

where $p^{n+\frac{1}{2}} = p^{n-\frac{1}{2}} + \phi$. Finally, the third step is

$$\rho(u^{n+1} - u^*) = -\delta t \nabla \phi.$$

We start by defining all the variables used in the variational formulations.

```

u = TrialFunction(V)
v = TestFunction(V)
u_ = Function(V, name="u")
u_s = Function(V, name="u_tentative")
u_n = Function(V)
u_n1 = Function(V)
p = TrialFunction(Q)
q = TestFunction(Q)
p_ = Function(Q, name="p")
phi = Function(Q, name="phi")

```

Next, we define the variational formulation for the first step, where we have integrated the diffusion term, as well as the pressure term by parts.

```

f = Constant(mesh, PETSc.ScalarType((
F1 = rho / k * dot(u - u_n, v) * dx
F1 += inner(dot(1.5 * u_n - 0.5 * u_n
F1 += 0.5 * mu * inner(grad(u + u_n),
F1 += dot(f, v) * dx
a1 = form(lhs(F1))
L1 = form(rhs(F1))
A1 = create_matrix(a1)
b1 = create_vector(extract_function_s

```

Next we define the second step

```

a2 = form(dot(grad(p), grad(q)) * dx)
L2 = form(-rho / k * dot(div(u_s), q)
A2 = assemble_matrix(a2, bcs=bcp)
A2.assemble()
b2 = create_vector(extract_function_s

```

We finally create the last step

```

a3 = form(rho * dot(u, v) * dx)
L3 = form(rho * dot(u_s, v) * dx - k
A3 = assemble_matrix(a3)
A3.assemble()
b3 = create_vector(extract_function_s

```

As in the previous tutorials, we use PETSc as a linear algebra backend.

```
# Solver for step 1
solver1 = PETSc.KSP().create(mesh.com
solver1.setOperators(A1)
solver1.setType(PETSc.KSP.Type.BCGS)
pc1 = solver1.getPC()
pc1.setType(PETSc.PC.Type.JACOBI)

# Solver for step 2
solver2 = PETSc.KSP().create(mesh.com
solver2.setOperators(A2)
solver2.setType(PETSc.KSP.Type.MINRES)
pc2 = solver2.getPC()
pc2.setType(PETSc.PC.Type.HYPRE)
pc2.setHYPREType("boomeramg")

# Solver for step 3
solver3 = PETSc.KSP().create(mesh.com
solver3.setOperators(A3)
solver3.setType(PETSc.KSP.Type.CG)
pc3 = solver3.getPC()
pc3.setType(PETSc.PC.Type.SOR)
```

Verification of the implementation compute known physical quantities

As a further verification of our implementation, we compute the drag and lift coefficients over the obstacle, defined as

$$C_D(u, p, t, \partial\Omega_S) = \frac{2}{\rho L U_{mean}^2} \int_{\partial\Omega_S} \rho \nu n \cdot \nabla$$

$$C_L(u, p, t, \partial\Omega_S) = -\frac{2}{\rho L U_{mean}^2} \int_{\partial\Omega_S} \rho \nu n \cdot$$

where u_{t_S} is the tangential velocity component at the interface of the obstacle $\partial\Omega_S$, defined as $u_{t_S} = u \cdot (n_y, -n_x)$, $U_{mean} = 1$ the average inflow velocity, and L the length of the channel. We use `UFL` to create the relevant integrals, and assemble them at each time step.

```
n = -FacetNormal(mesh) # Normal point
dObs = Measure("ds", domain=mesh, subdomain_id=1)
u_t = inner(as_vector((n[1], -n[0])), u)
drag = form(2 / 0.1 * (mu / rho * inner(u_t, u_t)), dObs)
lift = form(-2 / 0.1 * (mu / rho * inner(u_t, u_t)), dObs)
if mesh.comm.rank == 0:
    C_D = np.zeros(num_steps, dtype=P)
    C_L = np.zeros(num_steps, dtype=P)
    t_u = np.zeros(num_steps, dtype=n)
    t_p = np.zeros(num_steps, dtype=n)
```

We will also evaluate the pressure at two points, one in front of the obstacle, $(0.15, 0.2)$, and one behind the obstacle, $(0.25, 0.2)$. To do this, we have to find which cell contains each of the points, so that we can create a linear combination of the local basis functions and coefficients.

```
tree = bb_tree(mesh, mesh.geometry.dim)
points = np.array([[0.15, 0.2, 0], [0.25, 0.2, 0]])
cell_candidates = compute_collisions(points, mesh)
colliding_cells = compute_colliding_cells(points, mesh, cell_candidates)
front_cells = colliding_cells.links(0)
back_cells = colliding_cells.links(1)
if mesh.comm.rank == 0:
    p_diff = np.zeros(num_steps, dtype=P)
```

Solving the time-dependent problem

Stability of the Navier-Stokes

equation

Note that the current splitting scheme has to fulfil the a [Courant–Friedrichs–Lewy condition](#). This limits the spatial discretization with respect to the inlet velocity and temporal discretization. Other temporal discretization schemes such as the second order backward difference discretization or Crank–Nicholson discretization with Adams–Bashforth linearization are better behaved than our simple backward difference scheme.

As in the previous example, we create output files for the velocity and pressure and solve the time-dependent problem. As we are solving a time dependent problem with many time steps, we use the `tqdm`-package to visualize the progress. This package can be installed with `pip`.

```
from pathlib import Path

folder = Path("results")
folder.mkdir(exist_ok=True, parents=True)
vtx_u = VTWriter(mesh.comm, folder / "u.vtx")
vtx_p = VTWriter(mesh.comm, folder / "p.vtx")
vtx_u.write(t)
vtx_p.write(t)
progress = tqdm.autonotebook.tqdm(described)
for i in range(num_steps):
    progress.update(1)
    # Update current time step
    t += dt
    # Update inlet velocity
```

```

inlet_velocity.t = t
u_inlet.interpolate(inlet_velocity)

# Step 1: Tentative velocity step
A1.zeroEntries()
assemble_matrix(A1, a1, bcs=bcu)
A1.assemble()
with b1.localForm() as loc:
    loc.set(0)
assemble_vector(b1, L1)
apply_lifting(b1, [a1], [bcu])
b1.ghostUpdate(addv=PETSc.InsertMode, mode=vec_mode)
set_bc(b1, bcu)
solver1.solve(b1, u_s.x.petsc_vector)
u_s.x.scatter_forward()

# Step 2: Pressure correction step
with b2.localForm() as loc:
    loc.set(0)
assemble_vector(b2, L2)
apply_lifting(b2, [a2], [bcp])
b2.ghostUpdate(addv=PETSc.InsertMode, mode=vec_mode)
set_bc(b2, bcp)
solver2.solve(b2, phi.x.petsc_vector)
phi.x.scatter_forward()

p_.x.petsc_vector.axpy(1, phi.x.petsc_vector)
p_.x.scatter_forward()

# Step 3: Velocity correction step
with b3.localForm() as loc:
    loc.set(0)
assemble_vector(b3, L3)
b3.ghostUpdate(addv=PETSc.InsertMode, mode=vec_mode)
solver3.solve(b3, u_.x.petsc_vector)
u_.x.scatter_forward()

# Write solutions to file
vtx_u.write(t)
vtx_p.write(t)

# Update variable with solution from previous step
with (
    u_.x.petsc_vector.localForm() as loc_n,
    u_n.x.petsc_vector.localForm() as loc_n1,
    u_n1.x.petsc_vector.localForm() as loc_n1:
    loc_n.copy(loc_n1)
    loc_.copy(loc_n)

# Compute physical quantities
# For this to work in parallel, we need to go to processor zero and sum the contributions
drag_coeff = mesh.comm.gather(ass

```



```

lift_coeff = mesh.comm.gather(ass
p_front = None
if len(front_cells) > 0:
    p_front = p_.eval(points[0],
p_front = mesh.comm.gather(p_fron
p_back = None
if len(back_cells) > 0:
    p_back = p_.eval(points[1], b
p_back = mesh.comm.gather(p_back,
if mesh.comm.rank == 0:
    t_u[i] = t
    t_p[i] = t - dt / 2
    C_D[i] = sum(drag_coeff)
    C_L[i] = sum(lift_coeff)
    # Choose first pressure that
    for pressure in p_front:
        if pressure is not None:
            p_diff[i] = pressure[
            break
    for pressure in p_back:
        if pressure is not None:
            p_diff[i] -= pressure
            break
progress.close()
vtx_u.close()
vtx_p.close()

```

Solving PDE: 100% 12800/12800 [12:00<00:00, 1

Destroy PETSc objects to free memory

```

A1.destroy()
A2.destroy()
A3.destroy()
b1.destroy()
b2.destroy()
b3.destroy()
solver1.destroy()
solver2.destroy()
solver3.destroy()

```

<petsc4py.PETSc.KSP at 0x7fd01aecc3b0:

Verification using

data from FEATFLOW

As FEATFLOW has provided data for different discretization levels, we compare our numerical data with the data provided using

`matplotlib`.

```
if mesh.comm.rank == 0:
    if not os.path.exists("figures"):
        os.mkdir("figures")
    num_velocity_dofs = V.dofmap.index
    num_pressure_dofs = Q.dofmap.index

    turek = np.loadtxt("bdforces_lv4")
    turek_p = np.loadtxt("pointvalues")
    fig = plt.figure(figsize=(25, 8))
    l1 = plt.plot(
        t_u,
        C_D,
        label=r"FEniCSx ({0:d} dofs)".format(num_velocity_dofs),
        linewidth=2,
    )
    l2 = plt.plot(
        turek[1:, 1],
        turek[1:, 3],
        marker="x",
        markevery=50,
        linestyle="",
        markersize=4,
        label="FEATFLOW (42016 dofs)"
    )
    plt.title("Drag coefficient")
    plt.grid()
    plt.legend()
    plt.savefig("figures/drag_comparison.png")

    fig = plt.figure(figsize=(25, 8))
    l1 = plt.plot(
        t_u,
        C_L,
        label=r"FEniCSx ({0:d} dofs)".format(num_velocity_dofs),
        linewidth=2,
    )
    l2 = plt.plot(
        turek[1:, 1],
        turek[1:, 4],
        marker="x",
        markevery=50,
        linestyle="",

```

```

        markersize=4,
        label="FEATFLOW (42016 dofs)"
    )
    plt.title("Lift coefficient")
    plt.grid()
    plt.legend()
    plt.savefig("figures/lift_compari

fig = plt.figure(figsize=(25, 8))
l1 = plt.plot(
    t_p,
    p_diff,
    label=r"FEniCSx ({0:d} dofs)"
    linewidth=2,
)
l2 = plt.plot(
    turek[1:, 1],
    turek_p[1:, 6] - turek_p[1:,
    marker="x",
    markevery=50,
    linestyle="",
    markersize=4,
    label="FEATFLOW (42016 dofs)"
)
plt.title("Pressure difference")
plt.grid()
plt.legend()
plt.savefig("figures/pressure_com

```

